

(TR-104) PYTHON WITH AI/ML

Training Day 43 Report

Date: 11 March 2026

Topic Covered: Async RAG Pipeline Optimization, Secure Input Validation & Configuration Management

The forty-third day of training focused on optimizing the RAG-based AI Assistant by improving security validation, asynchronous execution, configuration management, and overall system maintainability for production-level deployment.

- Optimized banned keyword checking by converting keyword storage from list to frozenset for constant-time lookup performance.
- Improved keyword detection logic using generator expressions for cleaner and faster execution.
- Removed dependency on the bleach library and implemented a custom `validate_input()` function for request validation.
- Added regex-based validation to detect and reject XSS, HTML injection, prompt injection, and SQL injection attempts.
- Created `REJECT_PATTERNS` regex configuration for identifying dangerous inputs such as script tags, javascript injections, SQL queries, and prompt override attempts.
- Updated `ask.py` to use custom validation logic and removed stale imports and unused code.
- Added missing Redis and CORS configuration fields to `config.py` to prevent application startup errors.
- Created a `.env` file for securely storing configuration values and sensitive settings.
- Configured `config.py` using Pydantic `BaseSettings` for environment-based configuration management.
- Created and verified `.gitignore` rules to ensure `.env` files are excluded from GitHub repositories.
- Converted RAG pipeline and guardrail workflows into asynchronous functions using `async def`.

- Replaced synchronous `llm.invoke()` calls with `await llm.ainvoke()` for non-blocking LLM execution.
- Fixed coroutine handling inside the `rag_chain` pipeline for proper asynchronous execution flow.
- Reduced token count in safety prompts to optimize inference time during inline safety checks.
- Modified project folder structure for improved scalability and maintainability.
- Removed older deterministic keyword filtering logic after replacing it with more advanced validation-based security handling.
- Explored the deployment flexibility of Llama 3.2 as both a locally hosted and cloud-deployed language model.
- Strengthened understanding of asynchronous FastAPI workflows, secure AI system design, environment-based configuration handling, and optimized RAG pipeline execution.
- GitHub Link: https://github.com/JaspinderKaurWalia26/rag_pipeline
- Blog Link: <https://medium.com/@jaspinderkaurwalia855/building-a-rag-system-from-scratch-with-fastapi-chromadb-and-llama-3-2-80fbd5b0269e>